

PostgreSQL

Database Design Masterclass

From Zoho Analytics Tables → Normalized PostgreSQL Production Database

A complete step-by-step guide — from absolute beginner to confident database engineer

1. Install & Setup	2. Understand Your Data	3. Normalize	4. Design Schema	5. Build in PostgreSQL	6. Import Data	7. Indexes & Optimize
■ What You'll Learn		■■ Tools Covered		■ End Result		
Install PostgreSQL Normalize Zoho tables 1NF → 2NF → 3NF → BCNF Data types, constraints Import CSV/Excel/JSON Indexes & performance Backup & security		PostgreSQL 16 psql CLI tool pgAdmin 4 GUI DBeaver (free GUI) Python + psycopg2 CSV/Excel import SQL scripts		A fully normalized production-ready PostgreSQL database with proper foreign keys, indexes, constraints, and a clean data model you built yourself.		

■ TABLE OF CONTENTS — Your Learning Roadmap

Phase 1	Installing PostgreSQL & Essential Tools	3
Phase 2	Understanding Your Zoho Data — Audit & Inventory	6
Phase 3	Normalization — Clean Up Your Data Model	9
Phase 4	Designing Your PostgreSQL Schema	16
Phase 5	Creating Tables in PostgreSQL — All Commands	22
Phase 6	Importing Data — CSV, Excel, JSON, Zoho Export	30
Phase 7	Indexes, Constraints & Performance	36
Phase 8	Connecting Other Tools & Maintaining the Database	40
Quick Ref	Data Types, Commands & Troubleshooting Cheat Sheet	44

PHASE 1

INSTALLING POSTGRESQL & TOOLS

Setting Up Your Environment from Scratch

What is PostgreSQL?

PostgreSQL (often called Postgres) is a free, open-source relational database. Think of it as a very powerful spreadsheet engine that lives on your computer — it stores data in tables with rows and columns, but can handle millions of rows, complex relationships, and multiple users simultaneously. Zoho Analytics is built on top of a database like this internally.

The 3 Ways You Will Interact with PostgreSQL

1. `psql` — A command-line tool (like a text terminal). Type SQL directly. Fast for experts.
2. pgAdmin 4 — A visual browser-based tool. Point-and-click. Great for beginners.
3. DBeaver — A free desktop app. Best of both worlds. Recommended for day-to-day use.

Step-by-Step: Install PostgreSQL

1 Download PostgreSQL

Go to <https://www.postgresql.org/download/> and choose your operating system. Download the installer for PostgreSQL 16 (the latest stable version). The installer includes PostgreSQL, `psql`, and pgAdmin 4 — all in one.

2 Run the Installer

During installation you will be asked for:

Prompt	What to Enter	Notes
Installation directory	Leave as default	Usually C:/Program Files/PostgreSQL/16
Components	Check all boxes	PostgreSQL Server, pgAdmin, Stack Builder, Command Line Tools
Data directory	Leave as default	Where your actual data files will be stored
Password	Choose a strong password	WRITE THIS DOWN — you will need it always
Port	5432	This is the default. Leave it unless something conflicts
Locale	Default locale	Leave as default

■■ CRITICAL: Save Your Password

The password you set during installation is for the 'postgres' superuser. If you lose it, recovering access is complex. Write it down and store it safely.

3

Verify the Installation

Open your terminal / Command Prompt and run:

```
-- Windows: search for 'psql' in Start menu, or open Command Prompt:
psql -U postgres -h localhost

-- Mac: open Terminal and run:
psql -U postgres

-- Linux:
sudo -u postgres psql

-- You should see this prompt:
postgres=#

-- Type this to confirm it works:
SELECT version();

-- Then type \q to quit
```

4

Install DBeaver (Recommended GUI)

Download DBeaver Community Edition (free) from <https://dbeaver.io/download/>. Install it normally. When you open it, click 'New Database Connection', select 'PostgreSQL', enter Host: localhost, Port: 5432, Database: postgres, Username: postgres, and the password you chose.

5

Create Your First Database

In psql or DBeaver SQL editor — create your company database

```
-- Connect to PostgreSQL first (you're on the default 'postgres' database)
-- Now create your own database for the company:
CREATE DATABASE company_db;

-- Switch to it:
\c company_db

-- Confirm which database you are in:
SELECT current_database();

-- List all databases on your server:
\l
```

■ *Pro Tip: Always work inside your own database (company_db), never in the default 'postgres' database. Use \c company_db every time you open psql.*

PHASE 2

AUDITING YOUR ZOHU DATA

Understanding What You Have Before You Build

Why You Must Audit First

Before writing a single line of PostgreSQL code, you need to fully understand what data you currently have in Zoho Analytics. Skipping this step is the #1 mistake beginners make — they start building tables and then discover their data is messy, duplicated, or structured in a way that doesn't match their design.

Your Audit Checklist — Do This for EVERY Table in Zoho

- 1. What is the purpose of this table? What real-world thing does each row represent?
- 2. How many rows does it have?
- 3. What are all the columns? What data type is each one (text, number, date)?
- 4. Which column uniquely identifies each row? (This becomes your Primary Key)
- 5. Are there any columns repeated in multiple tables? (These become Foreign Keys)
- 6. Are there any columns that store multiple values in one cell? (Needs normalization)
- 7. Are there NULL values? How many? Are they expected or errors?
- 8. Are there obvious duplicates?

Export Your Zoho Tables to CSV

In Zoho Analytics, for each table: click the table → click Export → choose CSV format → download. Save all CSVs in a folder called 'zoho_export'. Do this for every single table.

A Real Example: What a Typical Zoho Setup Looks Like

Let's say your company has these Zoho tables. This is the messy 'before' picture that we will normalize and rebuild properly:

Zoho Table Name	Typical Columns (messy)	Problem
Sales Data	Customer Name, Customer Email, Customer Phone, Product, Category, Price, Qty, Total, Salesperson Name, Salesperson Email, Region, Date	Everything in one table — customers, products, staff all mixed
Contacts	Full Name, Email, Phone, Company, Company Address, Company Industry, Last Activity	Person and company data mixed together
Products	Product Code, Product Name, Category Name, Sub-Category, Unit Price, Cost	No separate categories table — categories repeated in every row
Invoices	Invoice No, Client Name, Client Email, Line Item 1, Line Item 2, Line Item 3, Total	Line items stored as separate columns — can't scale
Employees	Emp ID, Name, Dept Name, Manager Name, Manager Email, Salary	Manager data duplicated — no separate departments table

■ ■ These Are All Normalization Problems

Every 'Problem' listed above will cause data duplication, update anomalies, and reporting errors. Phase 3 will teach you exactly how to fix each one.

Document Your Entity Map

After auditing, write down the 'real-world things' your data is about. These become your tables in PostgreSQL. For most companies:

Real-World Entity	Becomes PostgreSQL Table	Identified By
A customer	customers	customer_id
A product	products	product_id
A sale / order	orders	order_id
Each line in an order	order_items	order_item_id
An employee	employees	employee_id
A department	departments	department_id
A supplier/vendor	vendors	vendor_id
An invoice	invoices	invoice_id
A category	categories	category_id
A region/location	regions	region_id

PHASE 3

NORMALIZATION

Cleaning Your Data Model — 1NF to BCNF

What is Normalization and Why Does It Matter?

Normalization is the process of organizing your database tables to eliminate data redundancy (storing the same thing multiple times) and ensure data integrity (no contradictions). Think of it like Marie Kondo for your database — each piece of information lives in exactly ONE place.

The Problems Normalization Solves

- Update Anomaly: Customer changes email → you must update 500 rows in Sales Data instead of 1 row in Customers.
- Insert Anomaly: You cannot add a product until someone actually buys it (if product data lives in Sales).
- Delete Anomaly: Deleting the last sale by a customer accidentally deletes all their contact info.
- Storage Waste: 'North Region' typed 10,000 times in Sales when it could be stored once.

First Normal Form (1NF) — One Value Per Cell

Rule: Every cell must contain ONE atomic (indivisible) value. No repeating groups. No comma-separated lists. No 'Item 1, Item 2, Item 3' columns.

BEFORE (Violates 1NF)	AFTER (Satisfies 1NF)
Invoice has: Line_Item_1, Line_Item_2, Line_Item_3 columns	Create a separate order_items table with one row per item
Tags column: 'urgent, important, followup'	Create a tags table with one tag per row, linked by ID
Phone: '555-1234, 555-5678' (two phones in one cell)	Create a contact_phones table with one phone per row
Skills: 'Python, SQL, Excel' in one column	Create an employee_skills table, one skill per row

1NF Fix: Invoices with multiple line items

```
-- WRONG (violates 1NF) - columns for each item:
-- invoices(invoice_id, customer, item1, qty1, item2, qty2, item3, qty3)

-- CORRECT (1NF) - separate table for line items:
CREATE TABLE invoices (
    invoice_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    invoice_date DATE NOT NULL
);

CREATE TABLE invoice_items (
    item_id SERIAL PRIMARY KEY,
```

```

    invoice_id INT NOT NULL,    -- links back to invoices
    product_id INT NOT NULL,
    quantity    INT NOT NULL,
    unit_price  NUMERIC(10,2) NOT NULL
);

```

Second Normal Form (2NF) — No Partial Dependencies

Rule: Every non-key column must depend on the **WHOLE** primary key, not just part of it. This only matters when your primary key is made of **MULTIPLE** columns (a composite key).

2NF Example — Order Items Table

Composite key: (order_id + product_id) together identify each row.

PROBLEM: product_name depends only on product_id, not the combination.

FIX: Move product_name to a separate products table — it belongs there.

Rule of thumb: If a column describes a part of the key, not the whole key, move it out.

Column in order_items	Depends on whole key?	Action
quantity	Yes — quantity of THIS product in THIS order	Keep in order_items
unit_price	Yes — price at time of THIS order	Keep in order_items
product_name	No — depends only on product_id	Move to products table
product_category	No — depends only on product_id	Move to products table
customer_name	No — depends only on order_id (not product_id)	Move to customers table

Third Normal Form (3NF) — No Transitive Dependencies

Rule: Non-key columns must depend **DIRECTLY** on the primary key, not on another non-key column. If Column C depends on Column B, and Column B depends on the primary key, that is a transitive dependency — fix it.

3NF Example — Employees Table

employees(emp_id, name, dept_id, dept_name, dept_budget)

emp_id → dept_id (employee's department ID)

dept_id → dept_name, dept_budget (transitive: dept name depends on dept_id, not emp_id)

FIX: Move dept_name and dept_budget to a separate departments table.

employees table keeps only dept_id as a foreign key reference.

3NF Fix: Split employees and departments

```

-- WRONG (violates 3NF): dept_name repeats in every employee row
-- employees(emp_id, name, dept_id, dept_name, dept_budget)

-- CORRECT (3NF): separate tables

```

```
CREATE TABLE departments (  
    dept_id    SERIAL PRIMARY KEY,  
    dept_name  VARCHAR(100) NOT NULL,  
    dept_budget NUMERIC(15,2)  
);  
  
CREATE TABLE employees (  
    emp_id     SERIAL PRIMARY KEY,  
    emp_name   VARCHAR(150) NOT NULL,  
    dept_id    INT REFERENCES departments(dept_id), -- FK only  
    salary     NUMERIC(10,2)  
);  
  
-- Now dept_name is stored ONCE. Change it in departments,  
-- and all 500 employees reflect the change automatically.
```

BCNF — Boyce-Codd Normal Form (Advanced)

Rule: Every determinant must be a candidate key. In practice, BCNF fixes edge cases in 3NF. For most company databases, reaching 3NF is sufficient. BCNF becomes relevant when a table has multiple overlapping candidate keys.

■ *Pro Tip: For 95% of real-world company databases, 3NF is your target. Don't over-normalize. Sometimes a slight denormalization for reporting performance is acceptable — but start with 3NF and denormalize intentionally later.*

Complete Normalization Example: Sales Data Table

Let's walk through normalizing the typical Zoho 'Sales Data' table completely:

STEP 1: Original messy table (everything in one place)

```
-- Original Zoho export columns:  
-- sale_id, date, customer_name, customer_email, customer_phone,  
-- customer_city, customer_country,  
-- product_name, product_category, unit_price,  
-- quantity, total_amount,  
-- salesperson_name, salesperson_email, region  
  
-- Problems:  
-- 1. Customer info repeated every sale (update anomaly)  
-- 2. Product info repeated every sale  
-- 3. Salesperson info repeated every sale  
-- 4. Region is just text – not linked to anything
```

STEP 2: After 1NF, 2NF, 3NF — proper normalized tables

```

-- Table 1: customers (customer data stored ONCE)
customers(customer_id PK, name, email, phone, city, country)

-- Table 2: regions
regions(region_id PK, region_name)

-- Table 3: employees / salespersons
employees(emp_id PK, name, email, region_id FK)

-- Table 4: categories
categories(category_id PK, category_name)

-- Table 5: products
products(product_id PK, name, category_id FK, unit_price)

-- Table 6: orders (one row per sale)
orders(order_id PK, order_date, customer_id FK, emp_id FK)

-- Table 7: order_items (one row per product per order)
order_items(item_id PK, order_id FK, product_id FK, quantity, unit_price)

-- Result: zero redundancy. Update a customer email in customers table.
-- All 10,000 orders instantly reflect the new email via the FK.

```

■ Notice that 'unit_price' stays in order_items (not just products). This is intentional — you want to record the price AT THE TIME of the sale, not the current price which may have changed later.

Normalization Decision Chart

Situation	Normalization Rule	Action
Multiple values in one cell	1NF	Create a child table, one row per value
Column depends on part of composite key	2NF	Move it to the table it actually describes
Column B depends on non-key Column A	3NF	Move Column B to a new table keyed by Column A
Customer data in sales table	3NF	Create a customers table, use customer_id FK in sales
Same category name in every product row	3NF	Create categories table, use category_id FK
Manager info in employee rows	3NF	employees.manager_id FK → employees.emp_id (self-join)
Address in orders AND customers	Design	Decide: store once in customers, or snapshot in orders?

PHASE 4

DESIGNING YOUR SCHEMA

Drawing Your Data Model Before Coding

What is a Schema?

A schema is the blueprint of your database — which tables exist, what columns each table has, what data types those columns use, and how tables relate to each other through foreign keys. Always design on paper (or a tool) before writing SQL.

PostgreSQL Data Types — Choose the Right One

Category	Data Type	Use For	Example
Whole Numbers	SMALLINT	Small integers -32768 to 32767	Age, Rating (1-10)
	INT / INTEGER	Standard integers up to ~2.1 billion	Quantity, Count
	BIGINT	Very large integers	Order numbers, User IDs (millions+)
	SERIAL	Auto-incrementing INT — use for Primary Keys	id SERIAL PRIMARY KEY
	BIGSERIAL	Auto-incrementing BIGINT	Large-scale PK
Decimals	NUMERIC(p,s)	Exact decimal — money, measurements	NUMERIC(10,2) for \$1234.56
	DECIMAL	Same as NUMERIC	Prices, percentages
	REAL	Approximate float (4 bytes)	Scientific measurements
	DOUBLE PRECISION	Approximate float (8 bytes)	High-precision measurements
Text	VARCHAR(n)	Variable text with max length n	name VARCHAR(200)
	TEXT	Unlimited length text	Notes, descriptions, addresses
	CHAR(n)	Fixed-length text — pads with spaces	country_code CHAR(2)
Dates/Times	DATE	Date only — no time	birth_date, hire_date
	TIME	Time only — no date	shift_start TIME
	TIMESTAMP	Date + time, no timezone	created_at TIMESTAMP
	TIMESTAMPTZ	Date + time WITH timezone (recommended)	order_placed TIMESTAMPTZ
	INTERVAL	A duration of time	INTERVAL '3 days'
Boolean	BOOLEAN	True/False values	is_active, is_paid
JSON	JSON / JSONB	Flexible data — JSONB is searchable	metadata JSONB
Other	UUID	Universally unique identifier	uuid UUID DEFAULT gen_random_uuid()

	ARRAY	Array of any type	tags TEXT[]
	ENUM	A fixed list of allowed values	status ENUM('active','inactive')

Choosing Between Data Types — Quick Rules

Money and Prices

ALWAYS use NUMERIC(10,2) for currency — never FLOAT or REAL.

FLOAT is approximate and will cause rounding errors in financial calculations.

NUMERIC(10,2) means: up to 10 digits total, with exactly 2 decimal places.

For very large amounts: NUMERIC(15,2). For micro-cents: NUMERIC(10,4).

IDs and Primary Keys

Use SERIAL for most tables (auto-generates 1, 2, 3...).

Use BIGSERIAL if you expect more than 2 billion rows (rare for most companies).

Use UUID if you need globally unique IDs across multiple systems or APIs.

NEVER use customer names, emails, or other real data as a primary key.

Text Columns

Use VARCHAR(n) when you have a known max length (names, codes, titles).

Use TEXT when length is truly unlimited (notes, descriptions, comments).

Avoid CHAR(n) unless the length is always exactly n characters (country codes).

Both VARCHAR and TEXT have identical performance in PostgreSQL — no speed difference.

Designing the Full Company Data Model

Here is the complete normalized schema for a typical company. Read the arrows as 'references':
order_items.product_id → products.product_id

Complete schema design (Entity-Relationship in text form)

regions

region_id PK

region_name

departments

dept_id PK

dept_name

region_id FK → regions.region_id

employees

emp_id PK

emp_name

email

```
dept_id      FK → departments.dept_id
manager_id   FK → employees.emp_id   (self-referencing!)
hire_date
salary

customers
customer_id  PK
full_name
email
phone
city
country
created_at

categories
category_id  PK
category_name
parent_category_id  FK → categories.category_id (self-ref for sub-cats)

vendors
vendor_id    PK
vendor_name
contact_email
country

products
product_id   PK
product_code  UNIQUE
product_name
category_id   FK → categories.category_id
vendor_id     FK → vendors.vendor_id
unit_price
cost_price
is_active

orders
order_id      PK
order_date
customer_id   FK → customers.customer_id
emp_id        FK → employees.emp_id   (salesperson)
status        ('pending','confirmed','shipped','delivered','cancelled')
```

```
notes

order_items
  item_id      PK
  order_id     FK → orders.order_id
  product_id   FK → products.product_id
  quantity
  unit_price   (snapshot of price at time of order)
  discount_pct

invoices
  invoice_id   PK
  order_id     FK → orders.order_id
  issued_date
  due_date
  is_paid
  paid_date
```

■ Draw this diagram using draw.io (free, at app.diagrams.net) or dbdiagram.io before writing any code. Seeing the relationships visually catches design errors early.

PHASE 5

CREATING TABLES IN POSTGRESQL

Complete SQL — Every Command You Need

The CREATE TABLE Command — Anatomy

Full CREATE TABLE syntax explained line by line

```
CREATE TABLE table_name (  
    column_name    data_type    constraints,  
    ...  
    -- table-level constraints at the bottom  
    CONSTRAINT constraint_name constraint_type (column)  
);  
  
-- Example fully annotated:  
CREATE TABLE employees (  
    emp_id        SERIAL        PRIMARY KEY,        -- auto-increment, unique, not null  
    emp_code      VARCHAR(10)    UNIQUE NOT NULL,    -- unique per company policy  
    emp_name      VARCHAR(200)    NOT NULL,          -- required  
    email         VARCHAR(255)    UNIQUE,            -- unique but optional  
    phone         VARCHAR(20),    -- optional  
    dept_id       INT            REFERENCES departments(dept_id) ON DELETE SET NULL,  
    manager_id    INT            REFERENCES employees(emp_id),    -- self-ref  
    hire_date     DATE            NOT NULL DEFAULT CURRENT_DATE,  
    salary        NUMERIC(10,2)   CHECK (salary >= 0),  
    is_active     BOOLEAN         NOT NULL DEFAULT TRUE,  
    created_at    TIMESTAMPTZ     NOT NULL DEFAULT NOW()  
);
```

All Constraint Types Explained

Constraint	SQL Syntax	Meaning	Example
PRIMARY KEY	col INT PRIMARY KEY	Uniquely identifies each row. Auto NOT NULL.	emp_id SERIAL PRIMARY KEY
NOT NULL	col TYPE NOT NULL	This column can never be empty/blank.	name VARCHAR(200) NOT NULL
UNIQUE	col TYPE UNIQUE	No two rows can have the same value.	email VARCHAR UNIQUE

DEFAULT	col TYPE DEFAULT val	If not provided, use this value.	status TEXT DEFAULT 'pending'
CHECK	col TYPE CHECK (expr)	Value must pass this condition.	salary NUMERIC CHECK (salary>=0)
FOREIGN KEY	col INT REFERENCES other(col)	Links to another table's PK.	dept_id INT REFERENCES departments(dept_id)
ON DELETE CASCADE	...REFERENCES t(c) ON DELETE CASCADE	Delete child rows when parent is deleted.	Used in order_items
ON DELETE SET NULL	...REFERENCES t(c) ON DELETE SET NULL	Set FK to NULL when parent is deleted.	Used in employees.manager_id
ON DELETE RESTRICT	...REFERENCES t(c) ON DELETE RESTRICT	Block deletion if children exist. Default.	Most FK relationships

Building All Tables — Complete Script

Run this script in order — dependencies must be created first

```
-- =====
-- COMPANY DATABASE SCHEMA – Run in this exact order
-- =====

-- 1. REGIONS (no dependencies)
CREATE TABLE regions (
    region_id    SERIAL PRIMARY KEY,
    region_name  VARCHAR(100) NOT NULL UNIQUE
);

-- 2. DEPARTMENTS
CREATE TABLE departments (
    dept_id      SERIAL PRIMARY KEY,
    dept_name    VARCHAR(100) NOT NULL UNIQUE,
    region_id    INT REFERENCES regions(region_id) ON DELETE SET NULL,
    budget       NUMERIC(15,2) DEFAULT 0
);

-- 3. EMPLOYEES (references departments – must come after)
CREATE TABLE employees (
    emp_id       SERIAL PRIMARY KEY,
    emp_code     VARCHAR(20) UNIQUE NOT NULL,
    emp_name     VARCHAR(200) NOT NULL,
    email        VARCHAR(255) UNIQUE,
    phone        VARCHAR(30),
    dept_id      INT REFERENCES departments(dept_id) ON DELETE SET NULL,
    manager_id   INT REFERENCES employees(emp_id) ON DELETE SET NULL,
```

```
hire_date    DATE NOT NULL DEFAULT CURRENT_DATE,
salary      NUMERIC(10,2) CHECK (salary >= 0),
is_active    BOOLEAN NOT NULL DEFAULT TRUE,
created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 4. CUSTOMERS
CREATE TABLE customers (
    customer_id SERIAL PRIMARY KEY,
    full_name    VARCHAR(300) NOT NULL,
    email        VARCHAR(255) UNIQUE,
    phone        VARCHAR(30),
    city         VARCHAR(100),
    country      VARCHAR(100) DEFAULT 'UAE',
    created_at   TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 5. CATEGORIES (self-referencing for sub-categories)
CREATE TABLE categories (
    category_id    SERIAL PRIMARY KEY,
    category_name   VARCHAR(150) NOT NULL UNIQUE,
    parent_category_id INT REFERENCES categories(category_id) ON DELETE SET NULL
);

-- 6. VENDORS
CREATE TABLE vendors (
    vendor_id      SERIAL PRIMARY KEY,
    vendor_name     VARCHAR(200) NOT NULL,
    contact_email   VARCHAR(255),
    phone           VARCHAR(30),
    country         VARCHAR(100)
);

-- 7. PRODUCTS
CREATE TABLE products (
    product_id     SERIAL PRIMARY KEY,
    product_code    VARCHAR(50) UNIQUE NOT NULL,
    product_name    VARCHAR(300) NOT NULL,
    category_id     INT REFERENCES categories(category_id) ON DELETE SET NULL,
    vendor_id       INT REFERENCES vendors(vendor_id) ON DELETE SET NULL,
```

```
    unit_price    NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    cost_price    NUMERIC(10,2) CHECK (cost_price >= 0),
    is_active     BOOLEAN NOT NULL DEFAULT TRUE,
    created_at    TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 8. ORDERS
CREATE TABLE orders (
    order_id      SERIAL PRIMARY KEY,
    order_date    TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    customer_id   INT NOT NULL REFERENCES customers(customer_id),
    emp_id        INT REFERENCES employees(emp_id) ON DELETE SET NULL,
    status        VARCHAR(20) NOT NULL DEFAULT 'pending'
                CHECK (status IN ('pending', 'confirmed', 'shipped', 'delivered', 'cancelled')),
    notes        TEXT
);

-- 9. ORDER ITEMS
CREATE TABLE order_items (
    item_id       SERIAL PRIMARY KEY,
    order_id      INT NOT NULL REFERENCES orders(order_id) ON DELETE CASCADE,
    product_id    INT NOT NULL REFERENCES products(product_id),
    quantity      INT NOT NULL CHECK (quantity > 0),
    unit_price    NUMERIC(10,2) NOT NULL CHECK (unit_price >= 0),
    discount_pct  NUMERIC(5,2) NOT NULL DEFAULT 0
                CHECK (discount_pct BETWEEN 0 AND 100)
);

-- 10. INVOICES
CREATE TABLE invoices (
    invoice_id    SERIAL PRIMARY KEY,
    order_id      INT NOT NULL UNIQUE REFERENCES orders(order_id),
    issued_date   DATE NOT NULL DEFAULT CURRENT_DATE,
    due_date      DATE NOT NULL,
    is_paid       BOOLEAN NOT NULL DEFAULT FALSE,
    paid_date     DATE,
    CHECK (due_date >= issued_date),
    CHECK (paid_date IS NULL OR paid_date >= issued_date)
);
```

Useful Table Management Commands

DDL commands you will use frequently

```
-- See all tables in your database
\dt

-- Describe a table structure
\d employees

-- Add a new column to existing table
ALTER TABLE customers ADD COLUMN loyalty_points INT DEFAULT 0;

-- Rename a column
ALTER TABLE customers RENAME COLUMN full_name TO customer_name;

-- Change a column's data type
ALTER TABLE products ALTER COLUMN unit_price TYPE NUMERIC(12,2);

-- Add a NOT NULL constraint to existing column
ALTER TABLE customers ALTER COLUMN email SET NOT NULL;

-- Remove a constraint
ALTER TABLE employees DROP CONSTRAINT employees_email_key;

-- Drop (permanently delete) a table
DROP TABLE IF EXISTS old_table;

-- Drop table and all tables that reference it
DROP TABLE categories CASCADE;

-- Rename a table
ALTER TABLE old_name RENAME TO new_name;

-- See all constraints on a table
SELECT conname, contype FROM pg_constraint WHERE conrelid='employees'::regclass;
```

PHASE 6

IMPORTING DATA

Bringing Your Zoho Data into PostgreSQL

The Import Strategy

Importing data is a multi-step process. You must import in the correct order — parent tables before child tables, because foreign key constraints will reject a row if the referenced parent doesn't exist yet.

Import Order	Table	Reason
1st	regions	No dependencies
2nd	departments	Needs regions
3rd	categories	Self-referencing, but parents before children
4th	vendors	No dependencies
5th	employees	Needs departments; managers must exist first
6th	customers	No dependencies
7th	products	Needs categories and vendors
8th	orders	Needs customers and employees
9th	order_items	Needs orders and products
10th	invoices	Needs orders

Method 1: COPY command (Fastest — for CSV files)

Import a CSV file using COPY

```
-- Basic import (file must be accessible by PostgreSQL server)
COPY customers (full_name, email, phone, city, country)
FROM '/path/to/customers.csv'
WITH (FORMAT CSV, HEADER TRUE, DELIMITER ',', NULL '');

-- On Windows, use forward slashes or double backslashes:
COPY customers (full_name, email, phone, city, country)
FROM 'C:/Users/YourName/zoho_export/customers.csv'
WITH (FORMAT CSV, HEADER TRUE);

-- \copy works from psql client (no server path needed - uses YOUR computer):
\copy customers (full_name, email, phone, city, country)
FROM 'customers.csv'
WITH (FORMAT CSV, HEADER TRUE);
```

```
-- Import with custom delimiter (tab-separated)
\copy products FROM 'products.tsv' WITH (FORMAT CSV, DELIMITER E'\t', HEADER);

-- Export a table to CSV
\copy customers TO 'customers_backup.csv' WITH (FORMAT CSV, HEADER TRUE);
```

■■ \copy (backslash copy) runs on the CLIENT machine (your laptop). COPY (without backslash) runs on the SERVER machine. Always use \copy when you're running psql on your local machine.

Method 2: pgAdmin Import (Point-and-Click)

In pgAdmin: Right-click on the table name → Import/Export Data → select Import, browse for your CSV file, set the delimiter and header options, click OK. This is the easiest method for beginners — no commands needed.

Method 3: INSERT statements (for small data or testing)

INSERT — add rows manually or from a script

```
-- Insert one row
INSERT INTO regions (region_name) VALUES ('North');
INSERT INTO regions (region_name) VALUES ('South');
INSERT INTO regions (region_name) VALUES ('East');
INSERT INTO regions (region_name) VALUES ('West');

-- Insert multiple rows at once (much faster)
INSERT INTO departments (dept_name, region_id, budget) VALUES
    ('Sales',      1, 500000.00),
    ('Marketing',  1, 300000.00),
    ('Engineering',2, 800000.00),
    ('HR',         3, 200000.00);

-- Insert and return the generated ID
INSERT INTO customers (full_name, email, phone, country)
VALUES ('Ahmed Al Rashid', 'ahmed@example.com', '+971501234567', 'UAE')
RETURNING customer_id;

-- Insert only if not already exists (upsert)
INSERT INTO categories (category_name)
VALUES ('Electronics')
ON CONFLICT (category_name) DO NOTHING;
```

Method 4: Python Script (for Excel and complex files)

Python script to import Excel/CSV files to PostgreSQL

```
# Install required libraries first:
# pip install psycopg2-binary pandas openpyxl
```

```
import pandas as pd

import psycopg2

from psycopg2.extras import execute_values

# Connect to PostgreSQL

conn = psycopg2.connect(

    host='localhost',

    database='company_db',

    user='postgres',

    password='your_password'

)

cur = conn.cursor()

# Read your Zoho CSV export

df = pd.read_csv('zoho_export/customers.csv')

# Clean the data (handle blanks, fix types)

df['email'] = df['email'].str.strip().str.lower()

df['phone'] = df['phone'].str.strip()

df = df.where(pd.notnull(df), None) # Convert NaN to None (NULL)

# Insert all rows efficiently

rows = list(df[['full_name', 'email', 'phone', 'city', 'country']].itertuples(index=False, name=None))

execute_values(cur,

    'INSERT INTO customers (full_name, email, phone, city, country) VALUES %s

    ON CONFLICT (email) DO NOTHING',

    rows

)

conn.commit()

print(f'Imported {len(rows)} customers')

cur.close()

conn.close()
```

Handling Dirty Data During Import

Problem in Zoho Export	PostgreSQL Error	Fix
Blank cells for required columns	NOT NULL violation	Use DEFAULT or clean in pandas before import
Duplicate emails	UNIQUE violation	ON CONFLICT (email) DO NOTHING or DO UPDATE
Date in wrong format (15/03/2024)	invalid input for date	Use TO_DATE() or pandas date parsing

Numbers with currency symbols (\$1,234)	invalid input for numeric	Strip symbols in pandas: <code>str.replace('\$','').replace(',','')</code>
Referencing IDs that don't exist yet	FK violation	Import parent tables first; check ID mapping
Text too long for VARCHAR(n)	value too long	Increase column length with ALTER TABLE or truncate
Mixed case duplicates (JOHN vs john)	No error — silent duplicate	Normalize with LOWER() before insert

Verifying Your Import

Run these checks after every table import

```
-- Count rows (compare with Zoho export row count)
SELECT COUNT(*) FROM customers;

-- Check for unexpected NULLs
SELECT COUNT(*) FROM customers WHERE email IS NULL;

-- Check for duplicates
SELECT email, COUNT(*) FROM customers
GROUP BY email HAVING COUNT(*) > 1;

-- Preview the data
SELECT * FROM customers LIMIT 10;

-- Check FK integrity (find orphaned rows)
SELECT oi.item_id FROM order_items oi
LEFT JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_id IS NULL;    -- should return 0 rows
```

PHASE 7

INDEXES, CONSTRAINTS & PERFORMANCE

Making Your Database Fast

What is an Index?

An index is like the index at the back of a textbook — instead of reading every page to find 'PostgreSQL', you jump straight to the right page. Without an index, every query scans every row (called a 'sequential scan'). With an index, PostgreSQL jumps directly to matching rows.

When to Create an Index

CREATE INDEX on columns you use frequently in WHERE clauses.

CREATE INDEX on Foreign Key columns (PostgreSQL does NOT auto-index FKs).

CREATE INDEX on columns used in JOIN ... ON conditions.

CREATE INDEX on columns used in ORDER BY when you query large tables.

Do NOT index every column — indexes slow down INSERT/UPDATE/DELETE slightly.

Rule of thumb: index columns that appear in WHERE or JOIN in your most common queries.

Creating indexes — all types

```
-- Basic B-tree index (default — works for =, <, >, BETWEEN, ORDER BY)
CREATE INDEX idx_orders_customer ON orders(customer_id);
CREATE INDEX idx_orders_date ON orders(order_date);
CREATE INDEX idx_products_category ON products(category_id);
CREATE INDEX idx_employees_dept ON employees(dept_id);

-- Composite index (for queries that filter on BOTH columns together)
CREATE INDEX idx_orders_customer_date ON orders(customer_id, order_date);

-- Unique index (enforces uniqueness AND creates an index)
CREATE UNIQUE INDEX idx_products_code ON products(product_code);

-- Partial index (index only active products — smaller, faster)
CREATE INDEX idx_active_products ON products(product_name) WHERE is_active = TRUE;

-- Case-insensitive text search index
CREATE INDEX idx_customers_email_lower ON customers(LOWER(email));

-- Then query with: WHERE LOWER(email) = LOWER('Ahmed@example.com')

-- List all indexes
\di

-- Drop an index
```

```
DROP INDEX idx_orders_customer;

-- See if PostgreSQL is using your index (query plan)
EXPLAIN ANALYZE SELECT * FROM orders WHERE customer_id = 42;

-- Look for 'Index Scan' – that means the index is being used.
-- 'Seq Scan' means PostgreSQL chose a full table scan instead.
```

Useful Database Maintenance Commands

VACUUM, ANALYZE, and maintenance

```
-- Update table statistics so PostgreSQL chooses good query plans
ANALYZE;                -- analyze all tables
ANALYZE customers;      -- analyze one table

-- Reclaim storage from deleted rows
VACUUM;                 -- vacuum all tables
VACUUM FULL customers;  -- full vacuum (locks table – run off-peak)

-- Combined (most common maintenance command)
VACUUM ANALYZE;

-- See table sizes
SELECT relname AS table,
       pg_size_pretty(pg_total_relation_size(relid)) AS total_size
FROM pg_catalog.pg_statio_user_tables
ORDER BY pg_total_relation_size(relid) DESC;

-- Find slow queries (requires pg_stat_statements extension)
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;
SELECT query, calls, total_exec_time, mean_exec_time
FROM pg_stat_statements
ORDER BY mean_exec_time DESC LIMIT 10;
```

PHASE 8

CONNECTING TOOLS & MAINTAINING

Backup, Security, and External Connections

Backup and Restore

Backup and restore commands — RUN IN TERMINAL, not psql

```
-- Backup entire database to a file
pg_dump -U postgres -d company_db -F c -f company_db_backup.dump

-- Backup as plain SQL (human-readable)
pg_dump -U postgres -d company_db > company_db_backup.sql

-- Backup a single table
pg_dump -U postgres -d company_db -t customers > customers_backup.sql

-- Restore from .dump file
pg_restore -U postgres -d company_db -F c company_db_backup.dump

-- Restore from SQL file
psql -U postgres -d company_db < company_db_backup.sql
```

Creating Read-Only Users (for Reports / Zoho Reconnect)

User and permission management

```
-- Create a read-only user for reporting tools
CREATE USER reports_user WITH PASSWORD 'strong_password_here';

-- Grant connection rights
GRANT CONNECT ON DATABASE company_db TO reports_user;

-- Grant read access to all current tables
GRANT USAGE ON SCHEMA public TO reports_user;
GRANT SELECT ON ALL TABLES IN SCHEMA public TO reports_user;

-- Grant access to future tables automatically
ALTER DEFAULT PRIVILEGES IN SCHEMA public
GRANT SELECT ON TABLES TO reports_user;

-- Create an app user with read+write (but no schema changes)
CREATE USER app_user WITH PASSWORD 'another_strong_password';
GRANT CONNECT ON DATABASE company_db TO app_user;
GRANT USAGE ON SCHEMA public TO app_user;
```

```
GRANT SELECT, INSERT, UPDATE, DELETE ON ALL TABLES IN SCHEMA public TO app_user;  
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO app_user;
```

Connecting Zoho Analytics Back to PostgreSQL

Once your PostgreSQL database is built and populated, you can connect Zoho Analytics to it as a live data source. In Zoho Analytics: Workspace → Import Data → From Database → PostgreSQL. Enter: Host (your server IP or localhost), Port (5432), Database (company_db), Username (reports_user), Password.

■ Use the read-only 'reports_user' when connecting Zoho Analytics. Never connect Zoho with the 'postgres' superuser — it's a security risk.

Connecting Python / Other Tools

Connection strings for common tools

```
-- Python (psycopg2)  
conn = psycopg2.connect('host=localhost dbname=company_db user=postgres password=xxx'  
)  
  
-- Python (SQLAlchemy)  
engine = create_engine('postgresql://postgres:password@localhost/company_db')  
  
-- Node.js (pg library)  
const pool = new Pool({ user:'postgres', host:'localhost',  
                        database:'company_db', password:'xxx', port:5432 })  
  
-- Power BI: use PostgreSQL connector, enter server:localhost, port:5432  
  
-- Tableau: Connect → PostgreSQL → server:localhost, port:5432  
  
-- DBeaver connection string:  
jdbc:postgresql://localhost:5432/company_db
```

PHASE REF

QUICK REFERENCE CHEAT SHEET

Commands, Types & Troubleshooting

■ Essential psql Commands

Command	What it Does
\l	List all databases
\c dbname	Switch to a database
\dt	List all tables in current database
\d tablename	Show table structure (columns, types, constraints)
\di	List all indexes
\du	List all users/roles
\timing	Toggle query execution time display
\e	Open last query in text editor
\i file.sql	Run a SQL file
\o file.txt	Save query output to a file
\copy	Import or export CSV (client-side)
\?	Help for backslash commands
\h CREATE TABLE	Help for a specific SQL command
\q	Quit psql
Ctrl+C	Cancel running query

■ Key SQL Commands Summary

Task	SQL Command
Create table	CREATE TABLE name (col type constraints);
Add column	ALTER TABLE name ADD COLUMN col type;
Rename column	ALTER TABLE name RENAME COLUMN old TO new;
Drop column	ALTER TABLE name DROP COLUMN col;
Change type	ALTER TABLE name ALTER COLUMN col TYPE newtype;
Delete table	DROP TABLE IF EXISTS name;
Insert rows	INSERT INTO name (cols) VALUES (vals);
Update rows	UPDATE name SET col=val WHERE condition;
Delete rows	DELETE FROM name WHERE condition;
Query data	SELECT cols FROM name WHERE condition ORDER BY col LIMIT n;

Import CSV	<code>\copy table FROM 'file.csv' WITH (FORMAT CSV, HEADER TRUE);</code>
Create index	<code>CREATE INDEX idx_name ON table(column);</code>
Create user	<code>CREATE USER name WITH PASSWORD 'pass';</code>
Grant access	<code>GRANT SELECT ON ALL TABLES IN SCHEMA public TO user;</code>
Backup	<code>pg_dump -U postgres -d dbname > backup.sql</code>
Restore	<code>psql -U postgres -d dbname < backup.sql</code>

■ Common Errors & Fixes

Error Message	Cause	Fix
ERROR: relation "X" does not exist	Table doesn't exist or wrong database	Run <code>\c company_db</code> to switch to correct database; check <code>\dt</code>
ERROR: duplicate key value violates unique constraint	Trying to insert a value that already exists	Use <code>ON CONFLICT DO NOTHING</code> , or clean data before import
ERROR: insert or update on table violates foreign key constraint	Referencing an ID that doesn't exist in parent table	Import parent table first; verify parent ID exists
ERROR: null value in column violates not-null constraint	Inserting NULL into a NOT NULL column	Add <code>DEFAULT</code> value or provide the value; check source data
ERROR: value too long for type character varying(n)	Text data exceeds column length	<code>ALTER TABLE ... ALTER COLUMN col TYPE VARCHAR(larger_n)</code>
ERROR: column X does not exist	Typo in column name, or wrong table	Run <code>\d tablename</code> to see exact column names
password authentication failed for user	Wrong password	Double-check password; reset with: <code>ALTER USER postgres WITH PASSWORD 'new';</code>
could not connect to server: Connection refused	PostgreSQL not running	Start service: <code>sudo service postgresql start</code> (Linux) or via Services on Windows

■ Full Phase-by-Phase Procedure Summary

Phase	What You Do	Key Commands / Tools
1. Install	Install PostgreSQL, pgAdmin, DBeaver	Installer from postgresql.org
2. Audit	Export all Zoho tables to CSV, document columns and types	Zoho Analytics → Export → CSV
3. Normalize	Apply 1NF, 2NF, 3NF rules to redesign tables on paper	Paper/whiteboard/draw.io
4. Design	Define all tables, data types, PKs, FKs, constraints	dbdiagram.io for ER diagram
5. Create	Write and run <code>CREATE TABLE</code> scripts in correct dependency order	<code>psql</code> or DBeaver SQL editor
6. Import	Import CSVs in parent-first order; verify with <code>COUNT</code> queries	<code>\copy</code> , pgAdmin Import, Python
7. Index	Create indexes on FK columns and frequently queried columns	<code>CREATE INDEX</code> statements

8. Connect	Create read-only user; connect Zoho/Power BI/Python to PostgreSQL	GRANT, CREATE USER
Ongoing	Regular backups, VACUUM ANALYZE, monitor performance	pg_dump, VACUUM ANALYZE

You Are Now a PostgreSQL Database Engineer!



Follow the 8 phases in order. Take your time on normalization — it's the most important step. A well-designed schema today saves you weeks of pain later. You now have everything you need.

— *Your Senior Developer*